

7-Point Next.js + Supabase Production Hardening Checklist

The exact architectural protocol engineered by Penn Enterprises LLC to eliminate silent database connection leaks, prevent RLS full-table scans, and guarantee sub-50ms query latency under heavy concurrency.

48-HR

SPRINT DELIVERY

< 50msTARGET QUERY
LATENCY**ZERO**PRODUCTION
DOWNTIME**50x**CONCURRENCY
SCALE

1 Route Serverless Connections Through Supavisor (Port 6543)

✗ Trap: Ephemeral serverless functions exhaust connections on port 5432, crashing with 'FATAL: remaining connection slots are reserved'.

✓ Fix: Set serverless DATABASE_URL to port 6543 (?pgbouncer=true). Retain port 5432 strictly for migrations.

```
DATABASE_URL="postgres://postgres:[REF]:[PW]@aws-0-[REGION].pooler.supabase.com:"
```

2 Cache Auth Evaluations in Row Level Security Policies

✗ Trap: `auth.uid()` inside policies re-evaluates per row, causing full-table scans and 1,200ms latency spikes.

✓ Fix: Wrap in `(select auth.uid())` to force Postgres to evaluate the session once per query.

```
CREATE POLICY "user_data_select" ON profiles FOR SELECT USING (user_id = (select
```

3 Enforce B-Tree Indexes on Foreign Keys and Tenant IDs

✗ Trap: Postgres does not automatically index foreign keys; unindexed `user_id` filters force Sequential Scans.

✓ Fix: Concurrently index all foreign keys referenced in RLS policies.

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_profiles_user_id ON public.profiles
```

4 Enforce Strict React Server Component (RSC) Data Boundaries

✗ Trap: Client-side `useEffect` fetches expose credentials, cause request waterfalls, and trigger hydration bugs.

✓ Fix: Keep database fetches strictly in Server Components; pass serializable data down to client leaves.

```
// app/(dashboard)/page.tsx (React Server Component)
const { data: metrics } = await supabase.from("metrics").select("*").limit(20);
```

5 Validate Data Mutations Through Zod-Gated Server Actions

✗ Trap: Raw client payloads allow field injection, type mismatch errors, or unauthorized privilege escalation.

✓ Fix: Validate all `formData` or `JSON` through strict `Zod` schemas inside Server Actions before writing.

```
const validated = ProfileSchema.safeParse(Object.fromEntries(formData));
if (!validated.success) throw new Error("Invalid payload");
```

6 Configure Global Statement Timeout Hard Stops

✗ Trap: A runaway query or lock can hang indefinitely, pegging CPU at 100% and taking down the database.

✓ Fix: Set a 5-second statement timeout on authenticated roles so runaway queries fail fast.

```
ALTER ROLE authenticated SET statement_timeout = '5000ms';
```

7

Empirical P99 Latency & Concurrency Verification

✗ Trap: Assuming code is ready because it runs locally with 1 user.

✓ Fix: Execute an isolated load test (200 concurrent users for 15s) to verify P99 TTFB < 50ms and 0 errors.

```
npx autocannon -c 200 -d 15s https://yourdomain.com/api/health
```

Penn Enterprises LLC • High-Performance AI & Backend
Engineering • San Diego, CA

While they wait on permission, we already
have the Penn. ⚡